

AI Application Architecture

How a request travels from the user to the AI model and back

Maryam Fatima | BS AI Student | Week 1

System Overview

This app is a single Flask application (`app.py`) that renders HTML pages directly — there is no separate frontend server. The browser submits a normal HTML form to Flask routes. Flask stores the running conversation in the session, calls the Gemini API, converts the reply to HTML with markdown, and then redirects back to the home route, which re-renders the page with the updated chat.

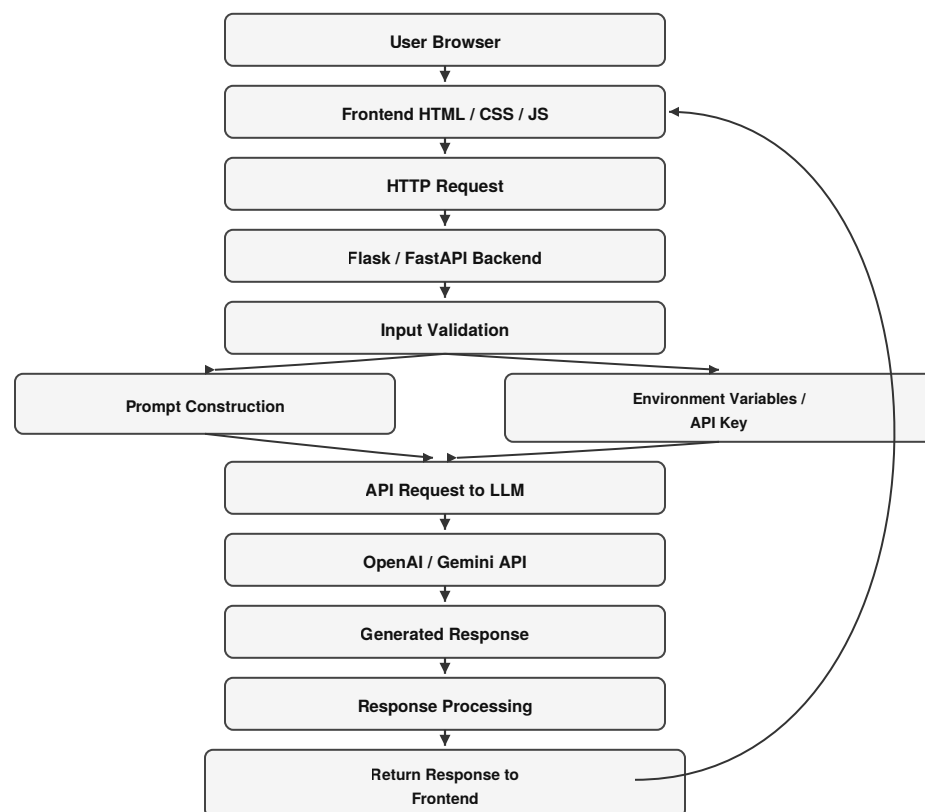


Fig. 1 — AI application architecture: request flows down, response loops back to the frontend.

Request Flow

The request flow follows the `/chat` route in `app.py`:

- **Browser:** the user fills the form with a prompt, an optional system prompt, and a selected model, then submits it with a POST request.
- **Flask App:** the `/chat` route reads the fields using `request.form.get()`. If the prompt is empty, it sets `session["error"]` and redirects back to home.
- **Session:** the user's message is appended to `session["messages"]` before the API call is made.

- **Gemini API:** the app calls `client.models.generate_content()`, passing the selected model, the prompt, and a `GenerateContentConfig` with the `system_instruction`.

Response Flow

Once Gemini replies, the rest of the `/chat` route handles the response:

- **Gemini API:** returns `response.text`, which becomes `raw_reply`.
- **Flask App:** converts `raw_reply` to HTML using `markdown.markdown()` with the `fenced_code` and `tables` extensions, then appends it to `session["messages"]` as the assistant's message.
- **Redirect:** the route calls `redirect(url_for("home"))`, sending the browser back to `/`.
- **Browser:** the `home()` route reads `session["messages"]` and passes them to `render_template("index.html", ...)`, showing the full updated chat.

Example: A user types "What is the weather today?" and submits the form. Flask stores this message, sends it to Gemini, gets a reply, converts it to HTML, stores it, and redirects to `/` so the browser shows the new message.

Error Handling

Error handling in this app happens in two places inside `app.py`:

- **Empty prompt:** in the `/chat` route, if `prompt` is empty after stripping whitespace, `session["error"]` is set to "Please enter a message." and the user is redirected home.
- **Gemini API failure:** the call to `client.models.generate_content()` is wrapped in a `try/except` block. If it fails, `ai_reply` is set to a fallback message telling the user to check their API key, internet connection, or selected model, and the real error is printed to the console with `print("Gemini API Error:", error)`.
- **Showing the error once:** the home route uses `session.pop("error", None)`, so the error message is shown one time and then cleared automatically.

API Integration

API integration is handled through the `google.genai` Python client:

- The client is created once at startup: `client = genai.Client(api_key=api_key)`, using a key loaded from the environment with `os.getenv("GEMINI_API_KEY")` and `load_dotenv()` — the key is never hardcoded.
- The model name is dynamic: it comes from the submitted form field `model`, defaulting to `"gemini-3.5-flash"` if none is selected, and is passed directly into `generate_content()`.
- The system prompt is sent using `types.GenerateContentConfig(system_instruction=system_prompt)`, so the AI's behavior can be changed per request.
- Two extra routes support the app: `/export` returns the full chat as a downloadable `.txt` file using Flask's `Response` object with a `Content-Disposition` header, and `/clear` removes `session["messages"]` to start a new conversation.

Week 1 Challenge — AI Application Architecture